📄 PART 1 — ARCHITECTURE & SETUP

🔹 Prompt 1

I am building an AI-powered pipeline failure monitoring system in Snowflake.

Before writing any SQL:

1. Identify all components required
2. Define schemas and their purpose
3. Define data flow from logs → AI → insights
4. Suggest Snowflake features to use (tasks, views, Cortex, etc.)
5. Highlight potential pitfalls

Constraints:

- Must use Snowflake Cortex COMPLETE()
- Must be fully automated using TASKS
- Should capture QUERY, TASK, and SYSTEM failures
- Must be near real-time (1–2 min latency)

Do NOT write SQL yet.
Return structured architecture only.

🔹 Prompt 2

Now create Snowflake setup based on approved architecture.

Context:
This system will monitor failures across pipelines and analyze them using AI.

Create:

1. Database: OPS_AI_MONITOR
2. Schemas:
    - EVENT_LOGS (raw failures)
    - AI_ENGINE (AI outputs)
    - METADATA (reference tables)
3. Warehouse:
    - Name: OPS_MONITOR_WH
    - Size: XSMALL
    - Auto suspend/resume enabled

Constraints:

- Use best naming conventions
- Fully qualified SQL
- Do NOT add unnecessary objects

Return ONLY executable SQL.

🔹 Prompt 3

Create a unified view that captures failures from multiple Snowflake metadata sources.

Sources:

1. INFORMATION_SCHEMA.QUERY_HISTORY
2. INFORMATION_SCHEMA.TASK_HISTORY
3. SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY

Requirements:

- Only FAILED queries/tasks

- Include:
  event_id, source_type, error_message, query_text,
  user_name, warehouse, database, schema, start_time

Constraints:

- Use UNION ALL
- Handle NULLs properly
- Limit recent data (last 5–10 mins for real-time)

Output:
CREATE VIEW SQL only.

📄 PART 2 — DATA PIPELINE + AI CORE

🔹 Prompt 4

Create a table to store all failure events.

Table: OPS_AI_MONITOR.EVENT_LOGS.FAILURE_EVENTS

Columns:

- event_id
- source_type
- error_message
- query_text
- user_name
- warehouse_name
- database_name
- schema_name
- start_time
- created_at (default current timestamp)

Constraints:

- Optimized for incremental ingestion
- Use appropriate data types

Return SQL only.

🔹 Prompt 5

Create a Snowflake TASK that runs every 1 minute.

Function:

- Merge new records from failure view into failure table
- Avoid duplicates using event_id

Constraints:

- Use MERGE
- Warehouse: OPS_MONITOR_WH
- Efficient incremental logic

Return:

1. CREATE TASK
2. MERGE logic
3. Resume statement

🔹 Prompt 6

Create a table to store AI analysis results.

Table: OPS_AI_MONITOR.AI_ENGINE.ERROR_ANALYSIS

Columns:

- event_id
- root_cause
- severity
- suggested_fix
- fix_sql
- confidence_score
- raw_response
- status (SUCCESS / FAILED)
- created_at

Constraints:

- Designed for Cortex JSON parsing
- Must support retries

Return SQL only.

🔹 Prompt 7

Generate Snowflake SQL that uses SNOWFLAKE.CORTEX.COMPLETE()

Context:
We are analyzing pipeline failures.

Input:

- error_message
- query_text

Output JSON format:
{
"root_cause": "",
"severity": "LOW|MEDIUM|HIGH",
"suggested_fix": "",
"fix_sql": "",

"confidence_score": 0.0
}

Rules:

- Always return valid JSON
- fix_sql must be executable
- Handle cases:
    - missing table
    - permission issues
    - syntax errors
    - division by zero

Constraints:

- Strip markdown formatting
- Use TRY_PARSE_JSON

Return ONLY SQL snippet.

🔹 Prompt 8

Create a Snowflake TASK that runs AFTER failure ingestion task.

Function:

- Pick latest unprocessed failures
- Call Cortex COMPLETE()
- Parse JSON
- Insert into AI table

Constraints:

- Limit to 10 records per run (cost control)
- Skip already processed events
- Handle NULL AI response

Return:

1. CREATE TASK
2. MERGE logic
3. Resume order instructions

📄 PART 3 — INSIGHTS + KB + ADVANCED FEATURES

🔹 Prompt 9

Create a final view combining:

- failure logs
- AI analysis

View: OPS_AI_MONITOR.AI_ENGINE.V_FAILURE_INSIGHTS

Output columns:

- event_id
- error_message
- query_text
- start_time
- root_cause
- severity
- suggested_fix
- fix_sql
- confidence_score
- status

Constraints:

- LEFT JOIN
- Optimized for dashboard usage

Return SQL only.

🔹 Prompt 10

Create an enriched view that classifies errors.

Add:

1. pipeline_type:
    - TASK, STORED_PROC, STREAM, DYNAMIC_TABLE, SQL
2. error_category:
    - MISSING_OBJECT
    - PERMISSION
    - SYNTAX
    - RUNTIME
    - OTHER

Use:

- QUERY_TEXT patterns
- ERROR_MESSAGE patterns

Return SQL for view.

🔹 Prompt 11 — Knowledge Base

I am enhancing an AI-powered Snowflake pipeline failure monitoring system.

Context:
We already capture failure logs and analyze them using Snowflake Cortex.

Now we want to improve AI accuracy using a knowledge base of common errors.

Task:
Create a reference table to store error patterns and recommended fixes.

Table:
OPS_AI_MONITOR.METADATA.ERROR_KB

Columns:

- pattern
- root_cause
- recommended_fix
- fix_sql

Insert sample rows:

- "does not exist"
- "object not found"
- "permission denied"
- "insufficient privileges"
- "division by zero"
- "warehouse is suspended"

Constraints:

- Use meaningful root cause and fix descriptions
- Include executable SQL in fix_sql
- Keep it simple and extensible

Output:
CREATE TABLE + INSERT SQL.

🔹 Prompt 12 — AI Queue

Create a view to identify pending failures for AI processing.

View:
OPS_AI_MONITOR.AI_ENGINE.V_AI_QUEUE

Logic:

- Select failures not yet analyzed OR failed previously
- Limit to recent records
- Include only required columns

Constraints:

- Avoid duplicates
- Limit to 20 records
- Optimize for task usage

Output:
CREATE VIEW SQL only.

🔹 Prompt 13 — KB Enhancement

Modify the AI processing logic to use knowledge base hints.

Context:
We have a table OPS_AI_MONITOR.METADATA.ERROR_KB.

Task:
Enhance Cortex prompt so that:

1. Match error_message with KB.pattern
2. Pass best matching KB.recommended_fix as a hint into prompt

Constraints:

- Keep JSON output format unchanged
- Ensure valid JSON parsing

- Must still work even if no KB match

Output:
Return ONLY updated SQL logic.

🔹 Prompt 14 — Documentation

Generate complete documentation for this system.

System Name:
AI Pipeline Failure Investigator

Include:

1. Architecture overview
2. Data flow
3. Snowflake objects
4. AI logic
5. Automation
6. Testing
7. Business value

Output:
Clean documentation text only.

🔹 Prompt 15 — Metrics

I want to add observability metrics to my pipeline monitoring system.

Create a metrics view:
OPS_AI_MONITOR.AI_ENGINE.V_FAILURE_METRICS

Metrics:

- total_failures
- query_failures
- task_failures
- dynamic_table_failures
- snowpipe_failures
- internal_errors
- high_severity_count
- success_count

Output:
CREATE VIEW SQL only.

🔹 Prompt 16 — SLA

I want to detect SLA breaches.

Expected runtime = 5 minutes

Create view:
OPS_AI_MONITOR.AI_ENGINE.V_SLA_BREACH

Output:
CREATE VIEW SQL only.

🔹 Prompt 17 — Custom Logs

I want to support external pipeline logs (Fivetran, DBT, custom ETL).

Create table:
OPS_AI_MONITOR.EVENT_LOGS.CUSTOM_LOGS

Add dummy data.

Output:
CREATE TABLE + INSERT SQL.

🔹 Prompt 18 — Alerts

Create a critical alerts detection system.

Conditions:

- HIGH severity
- SLA breach
- repeated failures

Output:
CREATE VIEW SQL only.

🔹 Prompt 19 — Auto Fix

Create automatic fix execution system.

- Read FIX_SQL
- Execute automatically

Conditions:

- Only LOW/MEDIUM severity
- Skip risky queries

Output:
CREATE TASK SQL.

Prompt 20 — Create Streamlit Dashboard

Create Streamlit Dashboard

📄 PART 4 — ASSISTANT + ADVANCED FEATURES

🔹 Prompt 21 — Improve CoCo Assistant Behavior

None

Improve CoCo Assistant behavior for analytical questions.

Context:
User asks questions like:
"How many dbt errors are in the account?"

Current Issue:
Assistant returns only SQL query.
This is NOT acceptable.

Requirements:
1. ALWAYS return FINAL ANSWER first (human-readable)
2. THEN optionally include SQL query used
3. NEVER return only SQL
4. If possible:
   - Execute logic conceptually and return count
   - Use existing views like:
     OPS_AI_MONITOR.AI_ENGINE.V_FAILURE_INSIGHTS

5. Format response as:

Answer:
<final number or insight>

Explanation:
<short reasoning>

SQL Used:
<query>

Example:

Question:
"How many dbt errors are there?"

Response:
Answer:
There are 24 dbt-related failures in the system.

Explanation:
These are identified using SOURCE_TYPE = 'DBT' in failure insights.

SQL Used:
SELECT COUNT(*)
FROM OPS_AI_MONITOR.AI_ENGINE.V_FAILURE_INSIGHTS
WHERE source_type = 'DBT';

Additional Rules:
- If exact data is not available → estimate + clearly mention assumption
- Prefer aggregated insights over raw SQL
- Keep response concise

Return updated assistant prompt or skill instructions.

🔹 Prompt 22 — Update skill.md Instructions

None

Update skill.md instructions:

1. AI Analysis Rules:
   - fix_sql is mandatory
   - never empty

```
   - fallback SQL required

2. Assistant Rules:
   - always answer first
   - SQL is secondary
   - no raw SQL-only responses

3. Output style:
   - structured
   - clean
   - production-ready

Return updated skill.md content.
```

📄 PART 5 — ENTERPRISE FEATURES (AIOps)

🔹 Prompt 23 — Slack / Teams Integration

```
SQL
Enhance the alerting system to support Slack and Microsoft Teams notifications in addition to email.

Context:
We already have SYSTEM$SEND_EMAIL working.

Goal:
Send real-time alerts to Slack/Teams when critical failures occur.

Requirements:
1. Create a notification integration for webhook-based alerts
2. Use external function or webhook to send payload
3. Message should include:
   - event_id
```

```
   - severity
   - pipeline_type
   - error_message (short)
   - suggested_fix
4. Format message as clean JSON

Slack Format:
{
  "text": "🚨 Pipeline Failure Alert",
  "attachments": [
    {
      "color": "danger",
      "fields": [
        {"title": "Severity", "value": "HIGH"},
        {"title": "Pipeline", "value": "DBT"},
        {"title": "Error", "value": "..."},
        {"title": "Fix", "value": "..."}
      ]
    }
  ]
}

Constraints:
- Must work with Snowflake (external function or procedure)
- Integrate with existing TSK_SEND_ALERTS

Return:
1. Integration design
2. SQL (external function / procedure)
3. Updated task logic
```

🔹 Prompt 24 — Human-in-the-Loop (HIL)

SQL

```
Design a Human-in-the-Loop (HIL) approval system for AI-generated fixes.

Context:
Currently, fixes are auto-executed.
```

We want human approval BEFORE execution.

Requirements:
1. Create new table:
   OPS_AI_MONITOR.AI_ENGINE.FIX_APPROVAL_QUEUE

Columns:
- event_id
- suggested_fix
- fix_sql
- severity
- confidence_score
- approval_status (PENDING / APPROVED / REJECTED)
- approved_by
- approved_at

2. Modify pipeline:
- AI generates fix → goes to approval queue
- DO NOT auto execute immediately

3. Auto-fix task should:
- Execute ONLY where approval_status = 'APPROVED'

4. Add logic:
- HIGH severity → always require approval
- MEDIUM → optional approval
- LOW → auto-approve allowed

Return:
1. CREATE TABLE SQL
2. Updated TSK_AUTO_FIX logic
3. Flow diagram (text)

🔹 Prompt 25 — Streamlit Approval UI

Python

Enhance Streamlit dashboard to support Human-in-the-Loop approval.

Context:
We have a Streamlit dashboard showing failures and fixes.

Requirements:
1. Add "Approve Fix" button per row
2. Add "Reject Fix" button
3. When clicked:
   - Update FIX_APPROVAL_QUEUE table
   - Set approval_status = APPROVED / REJECTED

4. Show:
   - Suggested fix
   - Fix SQL
   - Confidence score

5. UI behavior:
   - Disable button after action
   - Show status badge (Approved / Pending / Rejected)

6. Optional:
   - Add filter: show only pending approvals

Constraints:
- Use Snowpark session
- Use Streamlit best practices

Return:
Full Streamlit code snippet only

🔹 Prompt 26 — Post-Fix Validation

SQL

Design a post-fix validation system after AI executes a fix.

Goal:
After fix execution → validate system health automatically.

Requirements:

1. Create table:
   OPS_AI_MONITOR.AI_ENGINE.POST_FIX_VALIDATION

Columns:
- event_id
- validation_status (SUCCESS / FAILED)
- test_query
- test_result
- validation_time

2. After fix execution:
- Run validation queries:
   Examples:
   - SELECT COUNT(*) FROM affected table
   - Re-run failed query
   - Check object existence

3. Generate validation report:
- "Fix successful"
- "Fix partially successful"
- "Fix failed"

4. Store results in table

5. Optional:
- Send validation summary via alert

Constraints:
- Fully automated
- Must not break pipeline

Return:
1. Table SQL
2. Validation logic
3. Integration with TSK_AUTO_FIX

🔹 Prompt 27 — Auto Test Case Generation

SQL

Enhance AI system to generate test cases for each fix.

Context:
After generating fix_sql, we want AI to also generate validation queries.

Requirements:
Extend Cortex output JSON:

```
{
  "root_cause": "",
  "severity": "",
  "suggested_fix": "",
  "fix_sql": "",
  "test_sql": "",
  "confidence_score": 0.0
}
```

Rules:
1. test_sql must validate fix
2. Examples:
   - Missing table → SELECT COUNT(*) FROM table
   - Permission → SHOW GRANTS
   - Syntax → Re-run corrected query

3. test_sql must be safe and executable

4. Update parsing logic to store test_sql

Return:
Updated Cortex COMPLETE() prompt

🔹 Prompt 28 — Assistant Execute + Answer

SQL

Enhance CoCo AI Assistant to execute queries and return final answers instead of only SQL.

Context:

Currently assistant returns SQL like:
"SELECT COUNT(*) FROM ..."

This is not acceptable.

Requirements:
1. When user asks:
   "How many dbt errors in last 3 days?"

2. System should:
   - Generate SQL
   - EXECUTE internally
   - Return final answer

3. Response format:

Answer:
24 dbt failures in last 3 days

Explanation:
Based on SOURCE_TYPE = 'DBT'

SQL Used:
SELECT COUNT(*) FROM ...

4. Use:
OPS_AI_MONITOR.AI_ENGINE.V_FAILURE_INSIGHTS

5. NEVER return only SQL

6. If execution fails:
- return best estimate + explanation

Return:
Updated assistant prompt + logic

🔹 Prompt 29 — Smart Alerts with Approval

```
SQL
Enhance alerting system to include approval workflow.

Requirements:
1. Alert message should include:
   - Fix suggestion
   - Approve / Reject instruction

2. Format:
"Click to approve fix for EVENT_ID: XYZ"

3. If Slack:
- Include interactive button (if possible)
OR
- Include link to Streamlit dashboard

4. Email:
- Include event_id + instructions to approve in UI

5. Do NOT auto-fix until approved

Return:
Updated TSK_SEND_ALERTS logic
```

📄 PART 6 — SYSTEM FIXES (VERY IMPORTANT)

🔹 Prompt 30 — Full System Fix & Demo Readiness

```
None
You are an expert Snowflake + Streamlit + AI system engineer.

We have built an AI Pipeline Failure Investigator using Snowflake Cortex and CoCo.
The system is working, but we have multiple UX, logic, and consistency issues across tabs.
```

Your task is to:
1. FIX all issues
2. IMPROVE clarity for demo (non-technical stakeholders)
3. ENSURE consistency across all tabs
4. ADD missing fields (especially pipeline name)
5. PROVIDE updated SQL + Streamlit code changes

🔴 CRITICAL ISSUES TO FIX
---

1. OVERVIEW TAB - STATUS COLUMN
- Status shows "SUCCESS" for all records → unclear meaning
- Fix:
  - Clearly define what STATUS represents (AI success? pipeline success?)
  - Rename column if needed (e.g., AI_STATUS)
  - Add tooltip/description in UI

2. MISSING PIPELINE NAME (MAJOR ISSUE 🚨)
- Across ALL tabs:
  - Overview
  - Failure details
  - Fix approvals
- Currently only pipeline_type is shown
- Fix:
  - Add PIPELINE_NAME column
  - Extract from:
    - TASK → task name
    - STORED PROC → procedure name
    - DBT → model name
    - QUERY → query_text parsing
  - Display clearly in UI

3. FIX APPROVALS TAB ISSUES
- Clarify meanings:
  - "Executed"
  - "AUTO_APPROVED"
- Define logic:
  - AUTO_APPROVED → which severity? (LOW only?)
- Add:

  - Approval reason
  - Executed timestamp

4. APPROVE BUTTON ERROR
- Fix Streamlit approval button crash
- Ensure:
  - Proper DB update
  - No duplicate approvals
  - Error handling

5. SLA TAB
- Currently no breach example
- Fix:
  - Insert at least 1 demo SLA breach record
  - Ensure visible in UI

6. CRITICAL ALERTS TAB
- Explain clearly:
  - What are critical alerts?
  - Difference from normal failures
  - P1 / P2 / P3 definitions
- Add:
  - Fix recommendation
  - Approval button (same as Fix tab)

7. PIPELINE NAME VISIBILITY (GLOBAL FIX)
- Replace "pipeline_type only" with:
  - pipeline_name + pipeline_type
- Must appear in ALL tables

8. AI ANALYSIS TAB CONSISTENCY
- Ensure:
  - Same dataset as Overview & Fix tabs
  - No mismatch in counts

9. COCO AI TAB ISSUES
- Raw Data section unclear → explain meaning
- Fix chat UI:
  - Each Q&A should be separate
- Fix incorrect answer:
  - "medium severity auto_approved last 1 day" returning 0 but actual is 108

- Fix:
  - Query logic consistency with Fix_approvals table

10. ALERT INTEGRATION
- Add me to:
  - Email alerts
  - Slack alerts
- Ensure:
  - Alerts include pipeline_name + fix

11. POST-FIX VALIDATION + TESTING (DEMO GAP)
- We said it's implemented → need visible output
- Add:
  - Validation status column
  - Test results display
  - Example record for demo

12. AUTO TEST CASE GENERATION
- Show:
  - Generated test SQL
  - Execution result
- Add section in UI

🟢 IMPROVEMENTS REQUIRED
---

1. Add TOOLTIPS everywhere for business users:
- Status
- Severity
- Auto-approved
- Executed

2. Ensure consistent definitions across all tabs

3. Add DEMO DATA where missing:
- SLA breach
- Auto-approved fixes
- Validation results

4. Improve naming:

- Use business-friendly labels

🟡 OUTPUT REQUIRED

Return:

1. Updated SQL changes:
   - Views
   - Tables
   - Logic fixes

2. Updated Streamlit code:
   - UI fixes
   - Buttons
   - Tooltips
   - Chat improvements

3. Logic definitions:
   - STATUS meaning
   - AUTO_APPROVED rules
   - P1/P2/P3 explanation

4. Demo enhancements:
   - Sample data inserts
   - Validation examples

⚠️ CONSTRAINTS

- No breaking existing pipeline
- Maintain performance
- Keep UI clean and non-repetitive
- Ensure everything is demo-ready

🎯 GOAL

Make the system:
- Demo-friendly
- Business understandable
- Fully consistent
- Production-grade

Return complete solution.
